Lab # 3&4

DISCRETE TIME SYSTEM

Objective:

The objectives of the discrete time systems lab manual are:

1. To understand the basic concepts of discrete time signals and systems.
2. To learn how to analyze and design discrete time systems using mathematical tools such as convolution, Fourier transforms, and z-transforms.
3. To gain practical experience in implementing and simulating discrete time systems using software tools such as MATLAB or Python.
4. To learn how to design digital filters for signal processing applications.
5. To develop the ability to analyze and interpret the behavior of discrete time systems in both time and frequency domains.
6. To develop the ability to communicate effectively about discrete time systems and their applications.

Introduction

Discrete time systems are a fundamental topic in the field of signal processing and control systems. Discrete time systems are systems that operate on signals that are defined only at certain distinct points in time. These systems are widely used in digital signal processing, communication systems, and control systems.

The objective of the discrete time systems lab manual is to provide students with hands-on experience in the analysis and design of discrete time systems.

Classification of Discrete Time systems

A system is defined as an entity that acts on an input signal and transforms it into an output signal. A system may also be defined as a set of elements or functional blocks which are connected together and produces an output in response to an input signal. The response or output of the system depends on the transfer function of the system. It is a cause-and-effect relation between two or more signals. As signals, systems are also broadly classified into continuous-time and discrete-time systems. A continuous-time system is one which transforms continuous-time input signals into continuous-time output signals, whereas a discrete-time system is one which transforms discrete-time input signals into discrete-time output signals. For example, microprocessors, semiconductor memories, shift registers, etc. are discrete- time systems. A discrete-time system is represented by a block diagram as shown in Figure below. An arrow entering the box is the input signal (also called excitation, source or driving function) and an arrow leaving the box is an output signal (also called response). Generally, the input is denoted by x(n) and the output is denoted by y(n)

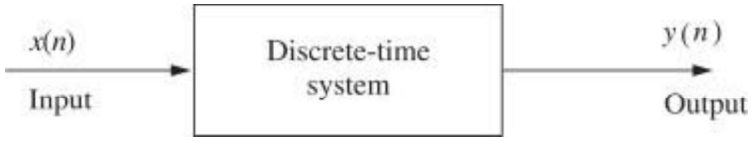
The relation between the input x(n) and the output y(n) of a system has the form:

$$y(n) = \text{Operation on } x(n)$$

Mathematically,

$$y(n) = T[x(n)]$$

which represents that x(n) is transformed toy(n). In other words, y(n) is the transformed. version of x(n).



Both continuous-time and discrete-time systems are further classified as follows:

- Static (memoryless) and dynamic (memory) systems
- Causal and non-causal systems
- Linear and non-linear systems
- Time-invariant and time varying systems

Static and Dynamic System systems

A system is said to be static or memoryless if the response is due to present input alone, i.e., for a static or memoryless system, the output at any instant n depends only on the input applied at that instant n but not on the past or future values of input or past values of output. For example, the systems defined below are static or memoryless systems.

$$y(n) = x(n)$$
$$y(n) = 2x2(n)$$

In contrast, a system is said to be dynamic or memory system if the response depends upon past or future inputs or past outputs. A summer or accumulator, a delay element is a discrete-time system with memory.
For example, the systems defined below are dynamic or memory systems.

$$y(n) = x(2n)$$
$$y(n) = x(n) + x(n-2)$$
$$y(n) + 4y(n-1) + 4y(n-2) = x(n)$$

Any discrete-time system described by a difference equation is a dynamic system.
A purely resistive electrical circuit is a static system, whereas an electric circuit having inductors and/or capacitors are a dynamic system. A discrete-time LTI system is memoryless (static) if its impulse response h(n) is zero.
for n not equal to 0. If the impulse response is not identically zero for n not equal to 0, then the system is called dynamic system or system with memory.

Example

```
import numpy as np
import scipy.signal as signal
# Define the system's transfer function or difference equation
num = [1, 2, 3]  # numerator coefficients
den = [1, 2, 1]  # denominator coefficients

# Check if the system is dynamic
is_dynamic = False
```

```
if np.roots(den).max() > 0:
    is_dynamic = True
# Print the result
if is_dynamic:
    print("The system is dynamic.")
else:
    print("The system is not dynamic.")
```

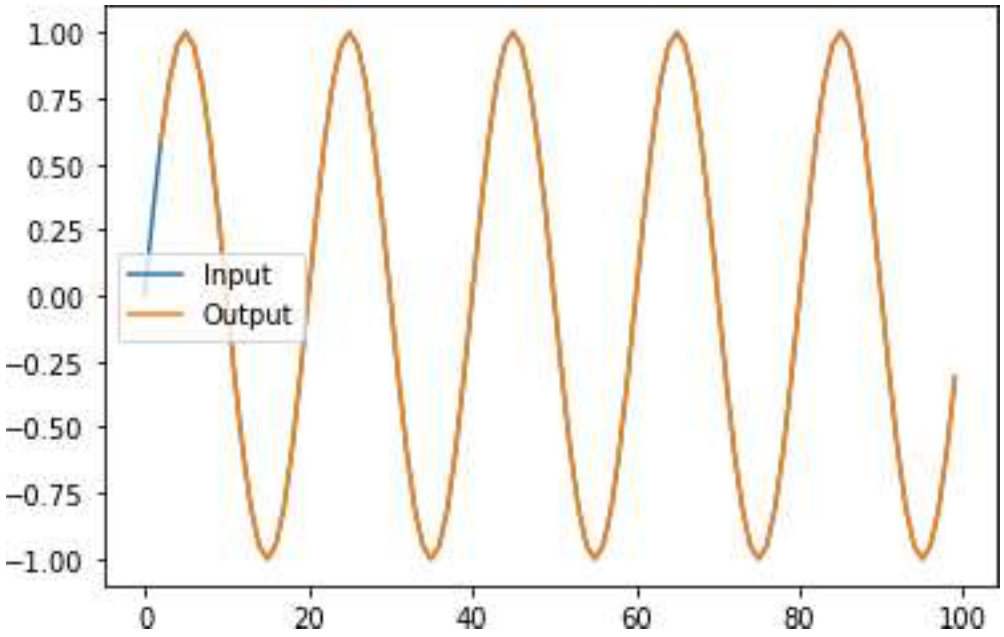
```
The system is not dynamic.
```

In this example, we first define the transfer function or difference equation of the system using the num and den coefficients. We then use the np.roots() function to find the roots of the denominator polynomial. If any of the roots have a positive real part, the system is dynamic, and we set the is dynamic flag to True. Finally, we print the result to the console.

Note that this code assumes that the system is represented in transfer function or difference equation form. If you have a different representation of the system, you may need to modify the code accordingly.

```
import numpy as np
import matplotlib.pyplot as plt

# Define the system's equation
system_eq = "y(n) = x(n+2)"

# Generate a test signal for the input
# For example, let's consider a sinusoidal input signal
n = np.arange(0, 100)
x = np.sin(0.1 * np.pi * n)

# Apply the system to the input signal
y = x[2:]

# Plot the input and output signals
plt.plot(n, x, label='Input')
plt.plot(n[2:], y, label='Output')
plt.legend()

# Check if the system is dynamic
is_dynamic = False
if np.any(np.diff(y) != 0):
    is_dynamic = True

# Print the result
```

```
if is_dynamic:
    print("The system {} is dynamic.".format(system_eq))
else:
    print("The system {} is not dynamic.".format(system_eq))
```



In this example, we define the system's equation using a string variable system_eq. We generate a test signal for the input (in this case, a sinusoidal signal) and apply the system to the input signal directly using array indexing to obtain the output signal y. We plot both the input and output signals using the matplotlib.pyplot module.

Next, we check if the system is dynamic by examining whether the output signal y changes over time. We do this by computing the differences between consecutive samples of the output signal using the np.diff() function and checking if any of the differences are non-zero. If there are non-zero differences, the system is dynamic, and we set the is_dynamic flag to True.

Finally, we print the result to the console. Note that in this example, the system is not dynamic because its output only depends on the current input sample and not on any past or future samples.

Causal and non-causal systems

A system is said to be causal (or non-anticipative) if the output of the system at any instant n depends only on the present and past values of the input but not on future inputs, i.e., for a causal system, the impulse response or output does not begin before the input function is

applied, i.e., a causal system is non anticipatory.

Causal systems are real time systems. They are physically realizable.

The impulse response of a causal system is zero for $n < 0$, since $s(n)$ exists only at $n = 0$,

i.e. h(n) = 0 for n < 0
The examples for causal systems are:
y(n) = nx(n)
y(n) = x(n – 2) + x(n – 1) + x(n)

A system is said to be non-causal (anticipative) if the output of the system at any instant n depends on future inputs. They are anticipatory systems. They produce an output even before the input is given. They do not exist in real time. They are not physically realizable. A delay element is a causal system, whereas an image processing system is a non-causal.
system.
The examples for non-causal systems are:

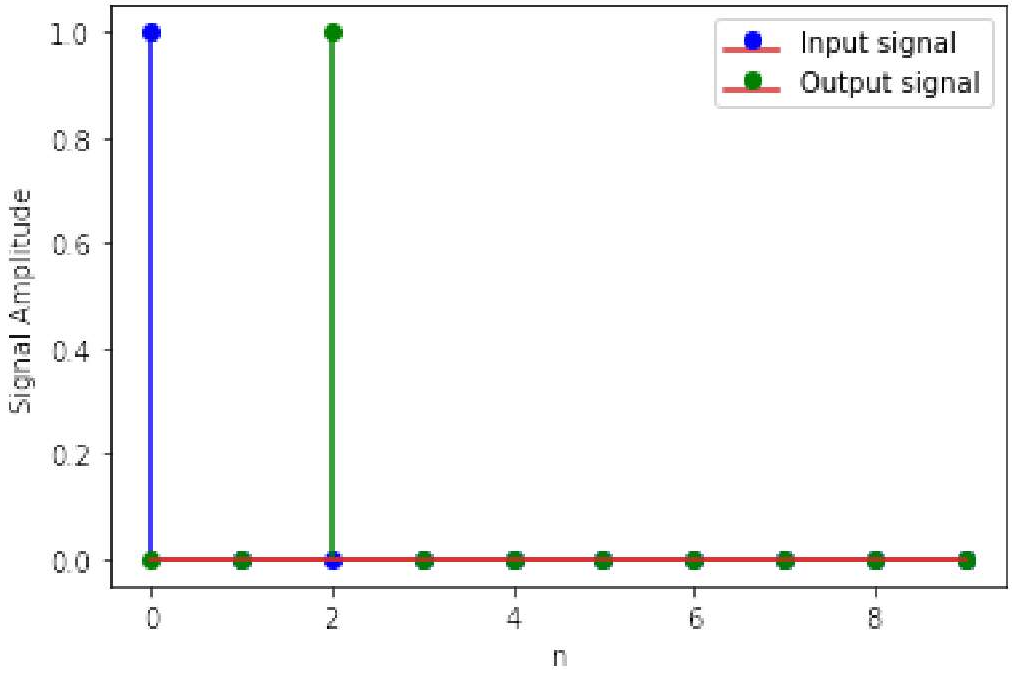
y(n) = x(n) + x(2n)
y(n) = x2(n) + 2x(n + 2)

```
# Example system coefficients
system = [2, 3, -1, 4]
# Check if the system is causal
is_causal = True
for i in range(1, len(system)):
    if system[i] < 0:
        is_causal = False
        break
if is_causal:
    print("The system is causal.")
else:
    print("The system is not causal.")
```

```
The system is not causal.
```

```
import numpy as np
import matplotlib.pyplot as plt

# Define the input signal
x = np.zeros(10)
x[0] = 1    # Unit impulse signal

# Compute the output signal
y = np.zeros_like(x)
for n in range(2, len(x)):
    y[n] = x[n] + x[n-2
# Check causality
if np.all(y[2:] == x[2:] + x[:-2]):
    print("The system is causal")
```

```
else:
    print("The system is non-causal")

# Plot the input and output signals
n = np.arange(len(x))
plt.stem(n, x, 'b', markerfmt='bo', label='Input signal')
plt.stem(n, y, 'g', markerfmt='go', label='Output signal')
plt.xlabel('n')
plt.ylabel('Signal Amplitude')
plt.legend()
plt.show()
```



Linear or nonlinear systems

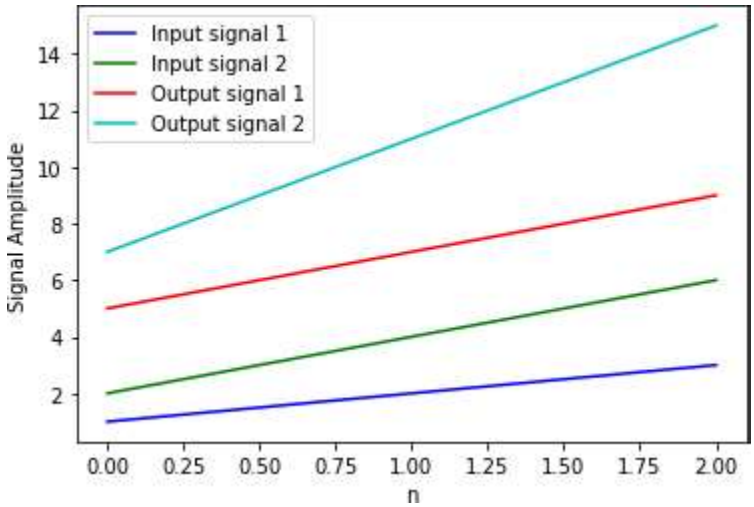
A system which obeys the principle of superposition and principle of homogeneity is called a linear system and a system which does not obey the principle of superposition an homogeneity is called a non-linear system. Homogeneity property means a system which produces an output y(n) for an input x(n) must produce an output ay(n) for an input ax(n).
Superposition property means a system which produces an output y1(n) for an input x1(n) and an output y2(n) for an input x2(n) must produce an output y1(n) + y2(n) for an input x1(n) + x2(n). Combining them we can say that a system is linear if an arbitrary input x1(n) produces an output y1(n) and an arbitrary input x2(n) produces an output y2(n), then the weighted sumof inputs ax1(n) + bx2(n) where a and b are constants produces an output ay1(n) + by2(n)
which is the sum of weighted outputs.

$$T(ax1(n) + bx2(n)] = aT[x1(n)] + bT[x2(n)]$$

Simply we can say that a system is linear if the output due to weighted sum of inputs is equal to the

weighted sum of outputs. In general, if the describing equation contains square or higher order terms of input and/or output and/or product of input/output and its difference or a constant, the system will definitely be non-linear.
Few examples of linear systems are filters, communication channels etc

```
# Compute the output signals
y1 = a*x1 + b          # Output signal for the first input
y2 = a*x2 + b          # Output signal for the second input

# Plot the input and output signals
plt.plot(x1, 'b', label='Input signal 1')
plt.plot(x2, 'g', label='Input signal 2')
plt.plot(y1, 'r', label='Output signal 1')
plt.plot(y2, 'c', label='Output signal 2')
plt.xlabel('n')
plt.ylabel('Signal Amplitude')
plt.legend()
plt.show()

if ((a*x1 + b) + (a*x2 + b) == a*(x1 + x2) + b).all():
    print("The system is linear")
else:
    print("The system is nonlinear")
```



Time-invariance is the property of a system which makes the behaviour of the system independent of time. This means that the behaviour of the system does not depend on the time at which the input is applied. For discrete-time systems, the time invariance property is called shift invariance.
A system is said to be shift-invariant if its input/output characteristics do not change with time, i.e., if a time shift in the input results in a corresponding time shift in the output

as shown in Figure 1.23, i.e.
If

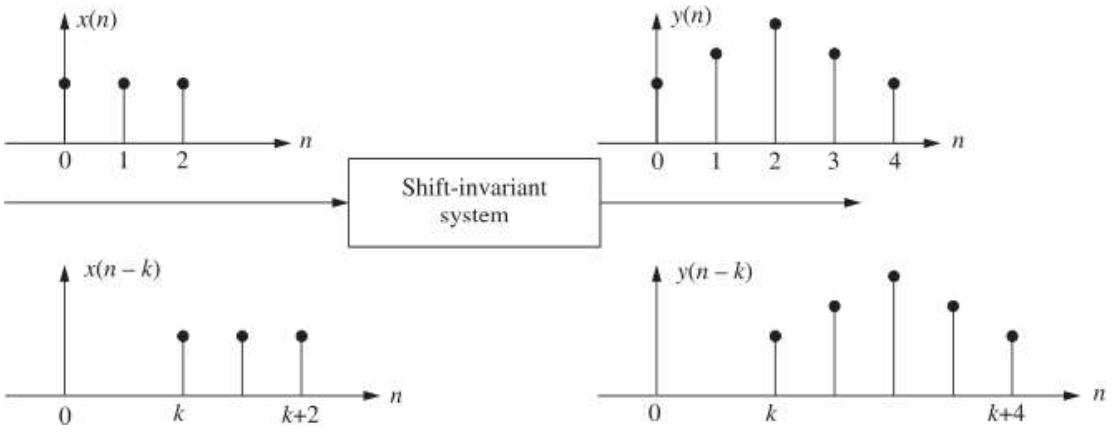
$$T[x(n)] = y(n)$$

Then

$$T[x(n-k)] = y(n-k)$$

A system not satisfying the above requirements is called a time-varying system (or shift-varying system). A time-invariant system is also called a fixed system.
The time-invariance property of the given discrete-time system can be tested as follows:
Let x(n) be the input and let x(n – k) be the input delayed by k units.

$y(n) = T[x(n)]$ be the output for the input x(n).



Time-invariant system

$y(n, k) = T[x(n-k)] = y(n)\big|_{x(n)=x(n-k)}$ be the output for the delayed input $x(n-k)$.

$y(n-k) = y(n)\big|_{n=n-k}$ be the output delayed by k units.

If y(n, k) = y(n – k)

i.e., if delayed output is equal to the output due to delayed input for all possible values of k, then the system is time-invariant.
On the other hand, if

y(n, k) not equal y(n – k)

i.e., if the delayed output is not equal to the output due to delayed input, then the system is time-variant. If the discrete-time system is described by difference equation, the time invariance canbe found by observing the coefficients of the difference equation.If all the coefficients of the difference equation are constants, then the system is time-invariant. If even one of the coefficients is function of time, then the system is time-variant.

The system described by

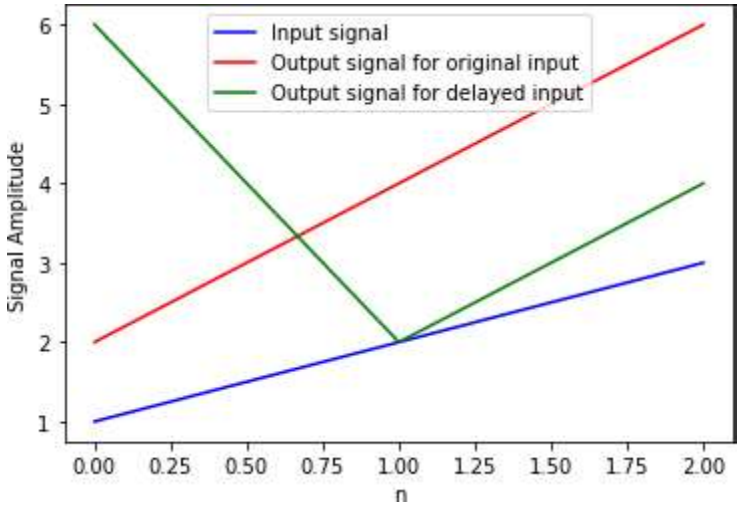
$$y(n) + 3y(n-1) + 5y(n-2) = 2x(n)$$

is a time-invariant system because all the coefficients are constants.
The system described by

$$y(n) - 2ny(n-1) + 3n2y(n-2) = x(n) + x(n-1)$$

is time-varying system because all the coefficients are not constant (Two are functions of time).
The systems satisfying both linearity and time-invariant conditions are called linear, time-invariant systems, or simply LTI systems.

```
import numpy as np
import matplotlib.pyplot as plt

# Define the input signal
x = np.array([1, 2, 3])    # Input signal

# Define the system
a = 2                      # System constant
n0 = 1                     # System delay

# Compute the output signals
y1 = a*x                   # Output signal for the original input
y2 = a*np.roll(x, n0)      # Output signal for the delayed input

# Plot the input and output signals
plt.plot(x, 'b', label='Input signal')
plt.plot(y1, 'r', label='Output signal for original input')
plt.plot(y2, 'g', label='Output signal for delayed input')
plt.xlabel('n')
plt.ylabel('Signal Amplitude')
plt.legend()
plt.show()

# Check time-invariance
if np.array_equal(a*x, a*np.roll(x, n0)):
   print("The system is time-invariant")
else:
   print("The system is time-varying")
```



In this code, we first define the input signal x. We then define a system with a constant a and a delay n0. We compute the output signals y1 and y2 by applying the system equation y(n) = ax(n - n0) to the input signal x and a delayed version of the input signal x obtained using the roll() function of the numpy library.

Next, we plot the input and output signals using the plot function of the matplotlib library.

Finally, we check time-invariance by comparing the output signals y1 and y2 obtained for the original input signal and the delayed input signal, respectively. If the outputs are the same for both input signals, the system is time-invariant; otherwise, it is time-varying. The code prints whether the system is time-invariant or time-varying.

In this case, the system is time-invariant.

Lab Tasks

Run all above codes

1. Find whether the following systems are dynamic or not:
   (a) y(n) = x(n + 2)
   (b) y(n) = x2(n)
   (c) y(n) = x(n – 2) + x(n)
2. Check whether the following systems are causal or not:
   (a) y(n) = x(n – 2) + x(n)
   (b) (b) y(n) = x(2n)
   (c) y(n) = sin[x(n)]
   (d) (d) y(n) = x(–n)

3. Check whether the following systems are linear or not:
   (a) (b) y(n) = x^2(n)
   (b) y(n) = x(n)+1/2x(n-2)
   (c) (d) y(n) = 2x(n)+4
   (d) (d) y(n) = x(n)coswn
   (e) y(n) = |x(n)|
4. Check whether the following systems are time-invariant or not:
   (a) y(n) = x(n/2)
   (b) y(n) = x(n)
   (c) y(n) = x^2(n-2)
   (d) y(n) = x(n)+nx(n-2)

Home task

Check whether the following systems are:

1. Static or dynamic
2. Linear or non-linear
3. Causal or non-causal, and
4. Shift-invariant or shift-variant

(a) $y(n) = \text{ev}\{x(n)\}$

(b) $y(n) = x(n)\,x(n-2)$

(c) $y(n) = \log_{10}|x(n)|$

(d) $y(n) = a^n u(n)$

(e) $y(n) = x^2(n) + \dfrac{1}{x^2(n-1)}$

Conclusion